Appendix: Items Corresponding to Second Edition

Second Edition Item Number	Third Edition Item Number, Title
1	1, Consider static factory methods instead of constructors
2	2, Consider a builder when faced with many constructor parameters
3	3, Enforce the singleton property with a private constructor or an enum type
4	4, Enforce noninstantiability with a private constructor
5	6, Avoid creating unnecessary objects
6	7, Eliminate obsolete object references
7	8, Avoid finalizers and cleaners
8	10, Obey the general contract when overriding `equals`
9	11, Always override `hashCode` when you override `equals`
10	12, Always override `toString`
11	13, Override `clone` judiciously
12	14, Consider implementing `Comparable`
13	15, Minimize the accessibility of classes and members
14	16, In public classes, use accessor methods, not public fields

15	17, Minimize mutability
16	18, Favor composition over inheritance
17	19, Design and document for inheritance or else prohibit it
18	20, Prefer interfaces to abstract classes
19	22, Use interfaces only to define types
20	23, Prefer class hierarchies to tagged classes
21	42, Prefer lambdas to anonymous classes
22	24, Favor static member classes over nonstatic
23	26, Don't use raw types
24	27, Eliminate unchecked warnings
25	28, Prefer lists to arrays
26	29, Favor generic types
27	30, Favor generic methods
28	31, Use bounded wildcards to increase API flexibility
29	33, Consider typesafe heterogeneous containers
30	34, Use enums instead of `int` constants
31	35, Use instance fields instead of ordinals
32	36, Use `EnumSet` instead of bit fields
33	37, Use `EnumMap` instead of ordinal indexing
34	38, Emulate extensible enums with interfaces

35	39, Prefer annotations to naming patterns
36	40, Consistently use the `Override` annotation
37	41, Use marker interfaces to define types
38	49, Check parameters for validity
39	50, Make defensive copies when needed
40	51, Design method signatures carefully
41	52, Use overloading judiciously
42	53, Use varargs judiciously
43	54, Return empty collections or arrays, not nulls
44	56, Write doc comments for all exposed API elements
45	57, Minimize the scope of local variables
46	58, Prefer for-each loops to traditional `for` loops
47	59, Know and use the libraries
48	60, Avoid `float` and `double` if exact answers are required
49	61, Prefer primitive types to boxed primitives
50	62, Avoid strings where other types are more appropriate
51	63, Beware the performance of string concatenation
52	64, Refer to objects by their interfaces
53	65, Prefer interfaces to reflection
54	66, Use native methods judiciously

55	67, Optimize judiciously
56	68, Adhere to generally accepted naming conventions
57	69, Use exceptions only for exceptional conditions
58	70, Use checked exceptions for recoverable conditions and runtime exceptions for programming errors
59	71, Avoid unnecessary use of checked exceptions
60	72, Favor the use of standard exceptions
61	73, Throw exceptions appropriate to the abstraction
62	74, Document all exceptions thrown by each method
63	75, Include failure-capture information in detail messages
64	76, Strive for failure atomicity
65	77, Don’t ignore exceptions
66	78, Synchronize access to shared mutable data
67	79, Avoid excessive synchronization
68	80, Prefer executors, tasks, and streams to threads
69	81, Prefer concurrency utilities to wait and notify
70	82, Document thread safety
71	83, Use lazy initialization judiciously
72	84, Don’t depend on the thread scheduler
73	(Retired)

74	85, Prefer alternatives to Java serialization 86, Implement `Serializable` with great caution
75	85, Prefer alternatives to Java serialization 87, Consider using a custom serialized form
76	85, Prefer alternatives to Java serialization 88, Write `readObject` methods defensively
77	85, Prefer alternatives to Java serialization 89, For instance control, prefer enum types to readResolve
78	85, Prefer alternatives to Java serialization 90, Consider serialization proxies instead of serialized instances

Previous chapter
< 12 Serialization

Next chapter
References >